# TenderAI: где мы сейчас и куда идём дальше

Постановка задачи, гипотезы, что уже закрыто в коде и что осталось

6 ноября 2026

## 1 Что мы решаем

На входе десять закупок и четыре артикула эластомерных инфузионных помп: `CT0050L`, `MC1248L`, `MS5248M` и `M275-07`. Нужно понять, какой артикул технически подходит под какую позицию закупки, и доказать это цитатами с обеих сторон. Цена, сроки, адрес поставки и документы участника в задачу не входят, только техника и комплектация.

Каждый артикул — это Excel с листом «Характеристики», всего 144 строки. Закупки устроены сложнее: 72 файла в форматах DOC, DOCX, XLS, XLSX, PDF, XML, TXT и RAR. Расширение иногда не совпадает с содержимым, три файла вместо документа содержат сохранённый ответ об ошибке ЕИС, а часть требований лежит не в таблицах, а в обычной прозе.

Ответ нужен на трёх уровнях: требование против артикула (900 пар), позиция против артикула (76 пар) и итоговая сводка «10 закупок на 4 артикула» (40 пар). На каждом уровне статус один из трёх.

`match.` Все применимые технические условия подтверждены, и у каждого есть источник.

`mismatch.` Доказано нарушение обязательного условия.

`unknown.` Всё остальное: значения нет, единицы несовместимы, требование не разобрано или в закупке битый файл ЕИС (`coverage_gap`).

У задания есть несколько жёстких правил, и почти все наши решения из них вытекают. Близость по смыслу не равна соответствию: числа, диапазоны, множества и условия «и/или» проверяет код. Характеристики соседней модели на артикул переносить нельзя. Пара, которую поиск отсёк, не становится `mismatch` автоматически. Отсутствие сведений о болюсе или УФ-защите не значит, что свойства нет.

В рабочем пайплайне участвует `gpt-5.6-luna` (через `cursor-sdk`), а расход токенов и вызовов пишется по этапам в `usage.jsonl`. Главный акцент оценки — латентные представления: как артикул и требование переводятся в общее векторное пространство и насколько это помогает сопоставлению по сравнению с простым поиском по словам.

## 2 Как мы проверяем идеи

Чтобы гипотеза что-то решала, нужен общий измеритель. Мы вручную проверили и закрыли отложенную выборку в `eval/held_out_draft.json`: 15 пар «требование на артикул» (`G01`–`G15`) по первичным файлам четырёх закупок и контрольный список `E01` из 12 прозовых требований первой позиции закупки 02. Каждую гипотезу проверяем одинаково: фиксируем вход, меняем один компонент и сравниваем метрики.

Метрики те же, что в задании: полнота и точность извлечения требований, `Recall@k` при поиске характеристики внутри SKU, `macro-F1` по трём статусам, точность класса `match`, доказательность цитат и токены по этапам.

## 3 Гипотезы и их текущий статус

Мы разбили их на четыре группы. Семь гипотез из десяти полностью закрыты и проверены в коде, а три онлайн-эксперимента ждут только свежего ключа `CURSOR_API_KEY`.

### Представление и латентное пространство

**H1. Кодировать нужно характеристику, а не изделие целиком.** Вектор на каждую характеристику против одного «мешка» на весь артикул. *Подтвердилась: убирает 533 ложных срабатывания.*

**H2. Латентное пространство FRIDA + RRF находит характеристику лучше лексики.** Плотные эмбеддинги `ai-forever/FRIDA` (1536d) с префиксами `search_query:` / `search_document:` и слиянием рангов через RRF($k = 60$) против лексического поиска. *Подтвердилась: точность выравнивания выросла с 0,667 до 0,933, `Recall@3` достиг 1,000, ложные `match` упали до 0, а на полном корпусе открылись 2 полных `match` закупок.*

**H3. Гибрид сохраняет точность там, где одна близость ошибается.** Статус по близости против схемы «поиск плюс проверка кодом». *Подтвердилась: отсекает 156 близких, но несовместимых пар.*

### Извлечение требований из грязных документов

**H4. GPT-Luna достаёт требования из прозы, которую не берёт табличный парсер.** *Подтвердилась: +118 требований, охват 10/10 закупок, на срезе `E01` точность и полнота 12/12.*

**H5. Поиск фрагментов до вызова модели экономит токены без потери требований.** Весь документ против top-$k$ релевантных страниц на закупках 02 и 03 (где полный документ 03 стоил 42 832 input-токенов). *Ждёт живой `CURSOR_API_KEY`.*

**H9. OCR нужен точечно, только там, где у PDF нет текстового слоя.** *Подтвердилась и подключена в коде через Apple Vision ровно для 1 пустого PDF из 8.*

### Кто выносит вердикт

**H7. Код не хуже модели на числах и при этом дешевле.** Кодовый чекер против вердикта `gpt-5.6-luna` на 15 проверенных парах. *Ждёт живой `CURSOR_API_KEY`.*

**H8. Каскад держит качество за долю токенов.** Сначала код за 0 токенов, вызов `gpt-5.6-luna` только на `unknown` (например, составное требование `G14`). *Ждёт живой `CURSOR_API_KEY`.*

**H10. Нормализация единиц и обработка отрицаний несут нагрузку.** *Подтвердилась: от отрицаний зависят 84 вердикта из 391, единицы страхуют 9 пар.*

### То, что мы отклонили

**H6. Реранкер поверх четырёх артикулов не окупается.** *Подтвердилась: мы проверяем все 4 SKU (Recall@4 = 1,0), реранкер отклонён.*

# 4 Что сделано и что показала проверка

## 4.1 Закрыты все три технических пробела в парсинге и два бага чисел

- **Закупка 09 (`02_Извещение.xml`):** файл оказался настоящим XLSX с расширением `.xml`. Передача `io.BytesIO` в `openpyxl` сняла отказ по расширению.
- **Закупка 05 (`02_5. пк.rar`):** подключена рекурсивная распаковка через `bsdtar`. Распаковка показала важный факт: внутри архива лежали только бланк проекта контракта и пустая таблица спецификации («не менее 6 месяцев» остаточного срока годности), а само внешнее ТЗ лежало в битом файле `08_Электронный документ...docx` (`saved_error_response`). Значит, статус `unknown` (`coverage_gap`) по закупке 05 абсолютно честный.
- **Закупка 04 (`06_Другое..`):** подключён гейтированный OCR на Apple Vision (`ocr.py`, кэш по SHA-256). Распознавание показало, что внутри одностраничный скан декларации по ПП РФ № 1875 об отсутствии российского производства, то есть ни одна техническая характеристика помпы в закупке 04 не потеряна.
- **Исправления в `parse.py`:** одиночные верхние неравенства (`<=`, $\leq$, `<`) теперь разбираются как верхняя граница `lte/lt` (раньше $\leq 2$ `мл` остаточного объёма ошибочно превращалось в `gt 2.0`), а записи вида `2:4:6:8:10:12:14` разбираются как числовое множество `set`.

В итоге в `runs/full/errors.json` осталось ровно 3 записи — три реальных JSON-ответа об ошибке выгрузки ЕИС в закупках 02, 05 и 06.

## 4.2 Ручная проверка отложенной выборки

Вычитка `eval/held_out_draft.json` глазами показала, почему нельзя доверять совпадению общих сумм статусов. В лексическом baseline распределение было таким же, как в эталоне (`7 match / 4 mismatch / 4 unknown`), но в `G11` скрывался ложный `match` (номинальный объём заполнения притянулся к максимальному 300 мл у 250-миллилитровой помпы `CT0050L`), в `G12` — потерянный `match` («Регулирование скорости: нет» не сматчилось с «Тип скорости: Постоянная»), а общая точность выравнивания составляла всего 10 из 15 (`0,667`). Подробный пообъектный разбор оформлен в `eval/REPORT.md` и в итоговом отчёте `docs/final_report.tex`.

## 4.3 Латентное пространство FRIDA и гибрид RRF (H2)

При подключении `ai-forever/FRIDA` (1536d) выяснилась принципиальная деталь: если класть в текст энкодера сырые числа и флаги «Да/Нет», в косинусном пространстве возникают межсвойственные коллизии — например, *«Скорость инфузии, мл/час: 5»* тянется к *«Объём болюса (мл): 5»* ($\cos = 0{,}391$) сильнее, чем к *«Настройки скорости: 2/4/6/8/10/12/14»* ($\cos = 0{,}174$).

Поэтому в `embeddings.py` мы кодируем предметно-оформленный **семантический слот** (`search_query: / search_document: Инфузионная помпа -- <характеристика[, ед. изм.][: категориальный текст]>`), а все числа, интервалы, множества и отрицания оставляем строго в `Constraint`. Ранг по косинусу FRIDA ($\geq 0{,}625$) объединяется с лексическим рангом через RRF($k = 60$) с фильтром метрологических квалификаторов (*минимальный / номинальный / максимальный / остаточный*).

| Метрика на `held_out_draft.json` (15 пар) | Лексика | FRIDA dense | Гибрид RRF |
|---|---|---|---|
| Точность выравнивания (`alignment_accuracy`) | 0,6667 | 0,8667 | **0,9333** |
| `Recall@1` / `Recall@3` (12 пар со слотом) | 0,8333 / 0,8333 | 0,9167 / 1,0000 | **0,9167 / 1,0000** |
| Отказ на отсутствующих слотах (`G08, G11, G13`) | 0 / 3 | 2 / 3 | **3 / 3** |
| `macro-F1` по статусам | 0,8690 | 0,9373 | **0,9373** |
| Доказательная точность `match` | 0,7143 | 1,0000 | **1,0000** |
| Ложные `match` | 1 (`G11`) | 0 | **0** |

### 4.4 Единый прогон по 10 закупкам и два подтверждённых `match`

В папке `runs/full` собран полный прогон с заполненными 1536-мерными векторами FRIDA в `representations/structured.json` (369 объектов). На 900 парах доля `not_aligned` снизилась с 45,2% (407) до **35,6% (320)**, а число разрешимых пар выросло с 391 до **445**.

Главное изменение на уровне позиций и закупок: если раньше подтверждённых совпадений было ноль, то теперь артикул `CT0050L` получил стопроцентно доказанный `match` **(7 из 7 требований)** сразу в двух закупках — **07** (`0372100048826000117`) и **10** (`0308300014226000025`). В обеих закупках латентное пространство дотянуло две недостающие характеристики: *«Путь введения: Внутривенный»* → *«Внутривенное введение: Да»* (cos = 0,7358) и *«Регулирование скорости инфузии: Нет»* → *«Тип скорости: Постоянная»* (cos = 0,6360). Код покрыт 37 автотестами, все проходят.

## 5 Что осталось доделать

Все оффлайн-артефакты сдачи (архитектура, таблица обоснования компонентов, таблица «baseline против итога», матрица $10 \times 4$, векторы в `runs/full` и сценарий демонстрации) уже собраны в `docs/final_report.tex`. При появлении активного ключа `CURSOR_API_KEY` остаётся прогнать три онлайн-замера с живыми вызовами `gpt-5.6-luna`:

1. **H5:** подать в `gpt-5.6-luna` только релевантные страницы закупок 02 и 03 и сравнить входные токены и полноту с полным прогоном (42 832 input-токенов на закупке 03).
2. **H7:** сравнить вердикт `gpt-5.6-luna` с кодовым чекером на 15 проверенных парах по `macro-F1` и расходу токенов.
3. **H8:** прогнать каскад «код за 0 токенов + `gpt-5.6-luna` только на `unknown`» (проверить, дорешивает ли каскад двухстрочный случай `G14` без потери точности).

#### Как воспроизвести текущее состояние

- Единый прогон (FRIDA + RRF + OCR + RAR + кэш Luna):
  `PYTHONPATH=src python -m tenderai run -dataset dataset_zakupki -out runs/full`
- Абляции H1, H2, H3, H6, H9 и H10:
  `python scripts/experiments.py`
- Автотесты (37 шт.): `pytest -v`
- Сводная таблица токенов: `python scripts/token_table.py`